

Retail Banking — Account, Transfers & Loan Servicing System

Summary:

Build a core retail banking module that manages customer accounts, deposits/withdrawals, internal transfers, and basic consumer loans. The system must ensure financial correctness, strict authorization, and transaction atomicity.

Core Features:

- CRUD: Customers, Bank Accounts, Transactions, Loans, Repayments, Account Statements.
- Authentication: Strong login (username/password), optional MFA, session/JWT tokens.
- Authorization / Roles: customer, teller, loan_officer, admin. Role-based access control.
- Transactional Support:
 - Transfers and withdrawals must be ACID: debit one account and credit another within the same DB transaction; on failure roll back both sides.
 - Loan disbursement should atomically create loan record and credit customer account.
 - Support distributed transaction patterns if microservices are used (sagas or two-phase commit alternatives).
- Validation & Business rules:
 - Prevent negative balances for non-overdraft accounts.
 - Enforce minimum balance, max transfer limits per day, KYC validations for account opening.

- Loan eligibility rules (income, credit score placeholder).
- Security & Data protection: encrypt sensitive data at rest (PII) and in transit; mask account numbers in UI; rate limiting on login and transfer APIs.
- Idempotency & Concurrency: idempotent transfer API (client-generated idempotency key), optimistic locking on balance updates to avoid race conditions.
- Audit & Logging: immutable transaction ledger, change history for critical operations, admin audit views, exportable statements.

New Feature — Database Locking for Multi-User Operations (Must Implement):

- Requirement: When a user performs a database-related operation that must be isolated (for example, balance update, loan disbursement, or large batch settlement), the system should place a lock so other users or processes cannot concurrently modify the same resource in a conflicting way.
- Lock types to consider (technology independent):
 - Pessimistic row-level locks (DB locks) for immediate exclusive access.
 - Optimistic locking with version numbers (detect conflicts and retry/abort).
 - Distributed locks (e.g., Redis Redlock, Zookeeper) for microservices or multi-node deployments.
- Lock semantics and requirements:

- Lock granularity: prefer row-level or resource-level locks (account_id, loan_id), not whole-database locks. Whole-database locks should be avoided unless explicitly required and justified.
- Lock scope: locks must protect critical sections such as debit-credit transfer, loan disbursement, and payroll payouts.
- Lock duration: locks must have TTL and must be released on success/failure; provide safe cleanup for orphaned locks (process crash).
- Deadlock handling: detect and resolve deadlocks; return clear error messages and support safe retry strategies.
- UI/UX: show "resource in use" or progress indicators when an operation is locked; prevent users from starting conflicting operations.
- Idempotency: operations should accept idempotency keys, and locks should be compatible with idempotent retries.
- Auditing: record lock acquisition/release events with user id, timestamp, and operation id.
- Acceptance tests:
 - Concurrent transfer attempts on same account must serialize correctly: final balances must be consistent.
 - Orphaned lock cleanup test: simulate crash and verify locks expire or are removed.
 - Distributed scenario: ensure lock works across multiple service instances (if

microservices used).